HTML Quick-Styler

Naan Mudhalvan Project Report Submitted to **Manonmaniam Sundaranar University** in the Partial fulfilment of the Requirement for the award of the **Degree of Bachelor of Science in Computer Science.**

Submitted By

TEAM ID: SWTID-2026-7065

TEAM LEADER: H. RABIATHUL AFRIN – 23082241802112035

TEAM MEMBER: S. PON ABINAYA – 23082241802112034

TEAM MEMBER: S.H. RABIYATHUL ATHABIYA – 23082241802112036

TEAM MEMBER: M.A. SAMU FAHEEMA – 23082241802112037

DEPARTMENT OF COMPUTER SCIENCE

WAVOO WAJEEHA WOMEN'S COLLEGE OF ARTS & SCIENCE

(Accredited by NAAC with 'B++' Grade)

KAYALPATNAM – 628 204

APRIL – 2025

CERTIFICATE

CERTIFICATE

This is to certify that the project work is entitled as **"HTML Quick-Styler" is a bonafide work by**

TEAM ID: SWTID-2026-7065

TEAM LEADER: H. RABIATHUL AFRIN – 23082241802112035

TEAM MEMBER: S. PON ABINAYA – 23082241802112034

TEAM MEMBER: S.H. RABIYATHUL ATHABIYA – 23082241802112036

TEAM MEMBER: M.A. SAMU FAHEEMA – 23082241802112037

In partial fulfilment of the requirement for the award of the degree of **BACHELOR OF COMPUTER SCIENCE** from **MANONMANIAM SUNDARNAR UNIVERSITY, TIRUNELVELI,** during the academic year of **2025-2026.**

Signature of the Incharge Staff **Signature of the H.O.D**

EXTERNAL EXAMINERS

1.
2.

Place: Kayalpatnam

Date:

DECLARATION

DECLARATION

We hereby declare that the Naan Mudhalvan Project, entitled **" HTML Quick-Styler"** submitted for the **Bachelor of Science in Computer Science** at **Manonmaniam Sundaranar University** is done. This project was completed by the **Department of Computer Science** at Wavoo Wajeeha Women's College of Arts and Science in Kayalpatnam.

Signature of the Candidate

1. **H. RABIATHUL AFRIN**
2. **S. PON ABINAYA**
3. **S.H. RABIYATHUL ATHABIYA**
4. **M.A. SAMU FAHEEMA**

DATE :

PLACE :Kayalpatnam

CONTENTS

S.NO	TABLE OF CONTENTS	PAGE NO
1	PROJECT DESCRPTION	8
2	SCENARIOS	10
3	A RCHITECTURE OVERVIEW	13
4	PROJECT WORKFLOW	16
5	PROJECT STRUCTURE	20
6	IMPLEMENTATION PHASE	22
7	CONCLUSION	44

PROJECT DESCRIPTION

Project Description

HTML Quick-Styler is an intelligent web page generation platform designed to simplify and accelerate the process of web development. The system leverages the powerful capabilities of IBM Granite 3.3 2B Instruct model to generate high-quality HTML and CSS code with professional styling and responsive design automatically.

The primary objective of HTML Quick-Styler is to address the challenges faced in rapid web development by providing an AI-powered environment capable of generating complete web page structures based on user requirements. The platform enables users to create modern and responsive web pages simply by providing basic inputs such as page title, section details, and color preferences.

Using the advanced natural language processing capabilities of the Granite 3.3 2B model, the system analyzes the user's description and intelligently generates a semantic HTML structure. It also applies modern CSS styling techniques, including responsive layouts, animations, and visually appealing color palettes to ensure that the generated pages follow current web design trends.

The system automatically detects different sections required in a webpage, such as headers, content areas, feature sections, and footers. It then organizes these sections into a clean and structured layout while maintaining proper HTML semantics and responsive behavior across different devices including desktops, tablets, and mobile screens.

To enhance usability, HTML Quick-Styler is implemented using a Gradio-based interactive interface that allows users to easily provide their inputs and instantly generate production-ready web page code. The platform also offers multiple color palette options and intelligent design suggestions to improve the visual appearance and usability of the generated pages.

Overall, HTML Quick-Styler transforms traditional web development into a more intelligent and user-friendly process by combining AI-driven content generation, automated design principles, and modern user interface technologies. The system ensures that the generated web pages follow professional design standards, accessibility guidelines, and responsive design practices, making it a powerful tool for developers, students, and designers who want to create high-quality websites quickly and efficiently.

SCENARIOS

Scenarios

Scenario 1: Portfolio Website Creation

A freelance designer intends to build a professional portfolio website in a quick and efficient manner. The system first understands the designer's requirements and analyzes the preferred design style and layout. Based on this information, it automatically generates a well-structured website layout.

The generated website begins with an attractive hero section that introduces the designer and includes a clear call-to-action to capture visitor attention. The system then creates a responsive portfolio gallery arranged in a grid layout so that projects can be displayed neatly across different screen sizes.

An About section is included to present the designer's background, skills, and professional experience in an engaging way. To enhance trust, the system also generates a testimonials section where client feedback can be showcased. A contact form with proper input validation is implemented to ensure smooth communication between visitors and the designer. Finally, the website is completed with a professional footer section containing social media links and important information.

Scenario 2: SaaS Product Landing Page

A startup company requires an effective landing page to introduce and promote their new software product. The system generates a visually attractive hero banner that clearly communicates the product's value and main purpose.

Next, a features section is created to present the main functionalities of the product using icons and short descriptions that make the benefits easy for users to understand. A pricing comparison table is also included so that visitors can easily compare different plans and choose the most suitable option.

To improve credibility, the system adds a testimonials section highlighting positive user experiences. Additionally, strategically placed call-to-action buttons encourage visitors to sign up, register, or purchase the product, helping increase conversions and user engagement.

Scenario 3: Corporate Business Website

A company needs a structured and professional website to represent its corporate identity. The system generates a clean and organized website layout starting with a sticky navigation header that includes the company logo and easy navigation links.

The website includes a company overview section that explains the organization's mission, vision, and core values. A services section is generated in a grid layout to clearly present the company's offerings.

To make the website more engaging, a team members section is added where employee photos and short profiles are displayed. A contact section is also implemented containing company details and a contact form that allows visitors to send inquiries directly. The website concludes with a professional footer section that contains privacy policies, quick links, and other important information.

Scenario 4: Color Palette Customization

A designer wants to explore various color combinations to identify the best visual theme for a website project. The system provides multiple pre-designed color palette options such as modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal themes.

Each palette includes clearly defined HEX color codes that allow easy implementation within the webpage design. The platform also provides a live preview feature, enabling the designer to instantly see how different color schemes affect the website's appearance.

The system ensures consistent color usage across all components of the webpage to maintain visual balance and harmony. Additionally, dark mode compatible backgrounds are supported to align with modern design preferences and improve user comfort during low-light viewing.

ARCHITECTURE OVERVIEW

Architecture Overview

HTML Quick-Styler is built as a modular platform combining Python backend with IBM Granite 3.3 2B Instruct model for intelligent HTML generation. The architecture prioritizes user experience, design quality, and code production through Gradio's interactive interface and sophisticated template generation.

Core Technologies

- Python 3.x: Core programming language for application logic
- IBM Granite 3.3 2B Instruct: AI model for intelligent content generation
- Gradio: Interactive web interface framework for user interaction
- HTML5/CSS3: Modern web standards for page generation
- JavaScript: Client-side interactivity and animations
- Google Colab: Cloud-based development environment
- Responsive Design: Mobile-first approach with media queries

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- Gradio: Install via pip (pip install gradio)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and model access

Knowledge Prerequisites

- Python Basics: Functions, classes, dictionaries, string manipulation
- HTML/CSS: Understanding of HTML structure and CSS styling
- Gradio Framework: Basic understanding of interface components
- Web Design: Color theory, responsive design principles
- Google Colab: Familiarity with notebook interface

Hardware Requirements

- Processor: Intel i3 or equivalent (Colab runs on cloud)
- RAM: Minimum 4GB local (Colab provides GPU/TPU)
- Storage: 1GB free space for generated files
- Display: Minimum 1280x720 resolution for interface

PROJECT WORKFLOW

Project Workflow

The development of the HTML Quick-Styler system is organized into multiple phases to ensure a structured implementation process. Each phase focuses on specific tasks that contribute to the overall functionality of the application.

Phase 1: Environment Setup and Model Initialization

This phase focuses on preparing the development environment and configuring the AI model required for the system.

Activity 1.1: Google Colab Environment Preparation

- Create a new notebook in Google Colab for project development.
- Activate GPU runtime to improve model processing performance.
- Install all required Python libraries needed for the project.
- Verify that the environment is correctly configured before continuing development.

Activity 1.2: IBM Granite Model Setup

- Access the model from the Hugging Face model repository.
- Load the Granite 3.3 2B Instruct model into the Colab environment.
- Perform basic tests to ensure that the model generates responses correctly.
- Adjust generation parameters to optimize output quality.

Activity 1.3: Gradio Installation and Configuration

- Install the Gradio library using the Python package manager.
- Run a simple Gradio interface to confirm installation success.
- Configure sharing options to allow external access if required.
- Test the interface to confirm proper rendering in the browser.

Phase 2: Core System Development

In this phase, the main logic of the system is implemented, including HTML generation and template management.

Activity 2.1: Color Palette Generator Development

- Design a ColorPaletteGenerator class to manage color schemes.
- Define multiple predefined color palettes for different design themes.
- Implement logic that allows the system to select palettes dynamically.
- Add validation methods to ensure color values are correctly formatted.

Activity 2.2: HTML Page Builder Implementation

- Develop an HTMLPageBuilder class responsible for generating page structure.
- Implement an algorithm that identifies the required page sections automatically.
- Create functions that generate structured HTML templates.
- Include a module to automatically generate SEO-friendly meta tags.

Activity 2.3: CSS Generation Engine Development

- Build a CSS generation system that styles all HTML components.
- Implement responsive design rules to support multiple screen sizes.
- Integrate animation and transition effects for improved visual appeal.
- Add mobile-first media queries to enhance responsiveness.

Activity 2.4: Section Template Creation

- Develop reusable templates for commonly used webpage sections.
- Implement a hero section template for landing introductions.
- Create a features grid layout for presenting product capabilities.
- Build a portfolio gallery template for showcasing projects.
- Add templates for contact forms, pricing tables, and testimonials.

Phase 3: Gradio User Interface Development

This phase focuses on designing an interactive interface that allows users to generate and preview web pages easily.

Activity 3.1: HTML Generation and Preview Interface

- A multi-tab user interface is created using Gradio to provide an intuitive workflow.
- The interface allows users to input webpage details, select color palettes, generate HTML code, and preview the final design in real time.
- This stage ensures that users can easily understand and visualize the generated web page structure without requiring advanced coding knowledge.

Phase 4: Google Colab Integration and Testing

In this phase, all components developed earlier are integrated and thoroughly tested.

Activity 4.1: System Integration Testing

- Integrate all application modules into a unified system.
- Verify communication between interface components.
- Test the entire HTML generation pipeline from input to output.
- Implement error handling to manage invalid inputs and unexpected behavior.

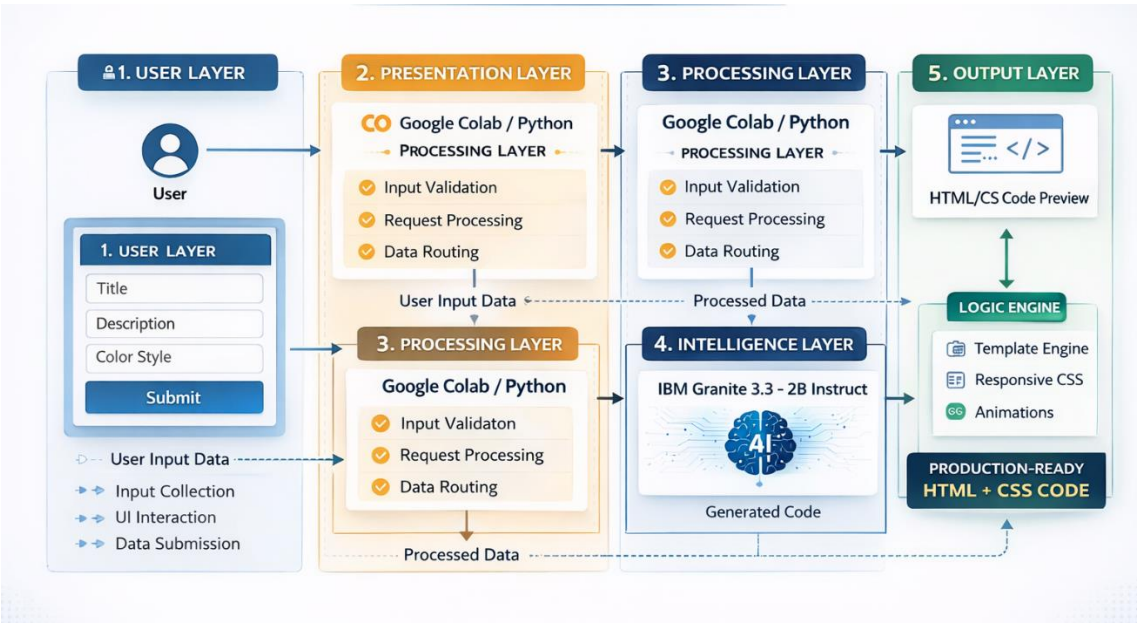
Phase 5: Deployment and Documentation

The final phase involves deploying the system and preparing project documentation.

Activity 5.1: Application Deployment and Functional Testing

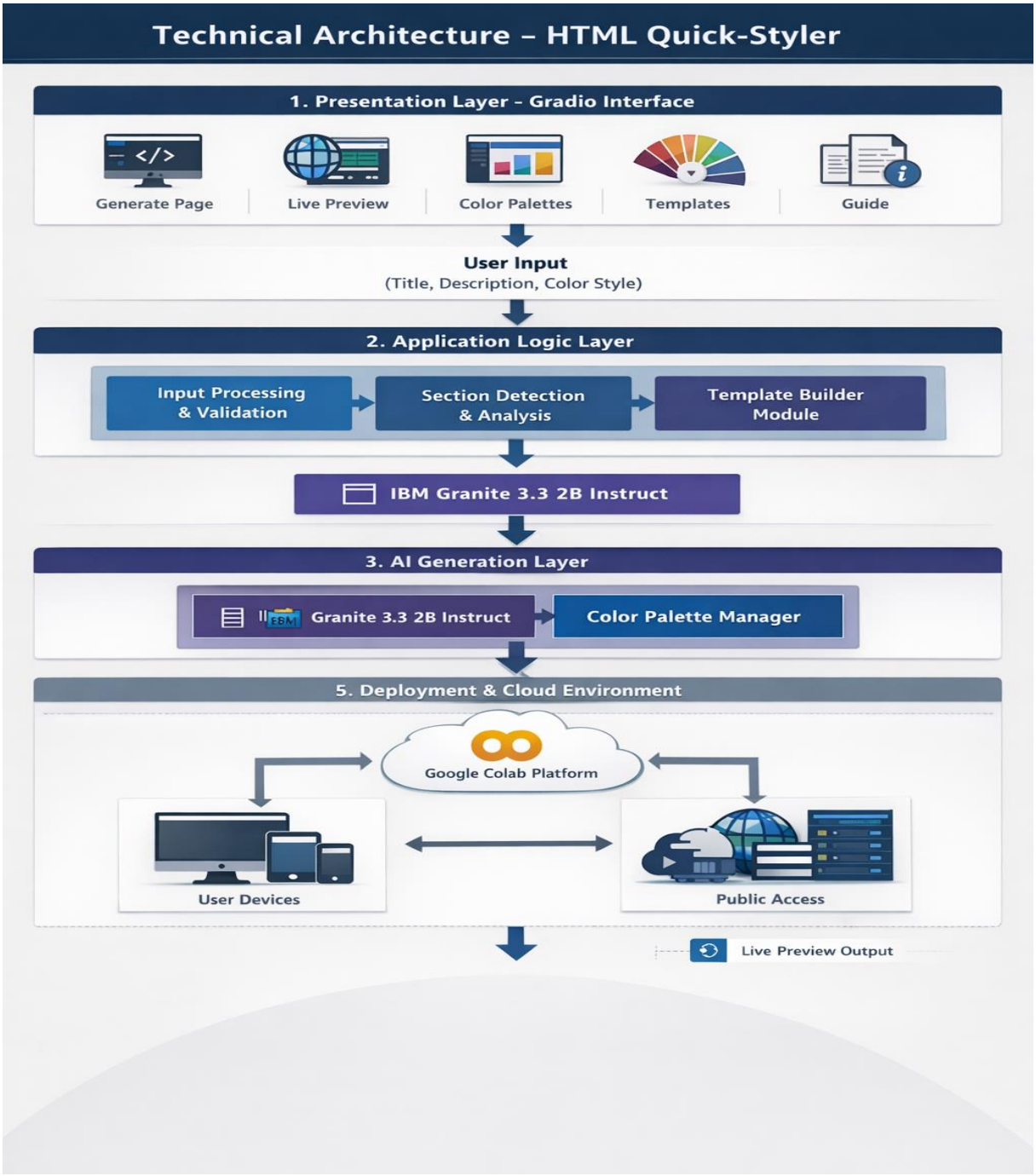
- Configure Gradio settings to allow public sharing of the interface.
- Generate a public access link for external users.
- Test the system from different devices and networks.
- Document the deployment procedure and system functionality for project reporting.• Test error handling

Technical Architecture



PROJECT STRUCTURE

Project Structure:



IMPLEMENTATION PHASE

Implementation Phase

PHASE 1: Environment Setup & Model Configuration

Establish Google Colab environment and configure Granite model

Activity 1.1: Install Dependencies & Setup Environment

Create requirements.txt:

```
[ ]
!pip install -q gradio

import gradio as gr
import json
import re
from datetime import datetime
from typing import Tuple, Dict, List
import random
```

Activity 1.2: Configure IBM Granite Model

Access Hugging Face and load model:

Model: ibm-granite/granite-3.3-2b-instruct

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
  messages,
  add_generation_prompt=True,
  tokenize=True,
  return_dict=True,
  return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))

Loading weights: 100% 362/362 [00:04<00:00, 139.69it/s, Materializing param=model.norm.weight]
```

Colab Notebook:

PHASE 2: Core Application Development

Activity 2.1: Color Palette Generator Implementation

Create ColorPaletteGenerator class with 8 predefined color schemes:

The ColorPaletteGenerator class provides cohesive color palettes for different design styles including modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal. Each palette includes primary, secondary, accent, background, surface, text primary, text secondary, and border colors optimized for dark theme interfaces.

```python
# HTML QUICK-STYLER - ADVANCED VERSION (FIXED)
# ==============================================================================

class ColorPaletteGenerator:
    """Generate visually cohesive color palettes"""

    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        """Generate a cohesive color palette based on style"""

        palettes = {
            'modern': {
                'primary': '#667eea',
                'secondary': '#764ba2',
                'accent': '#f093fb',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'vibrant': {
                'primary': '#ff6b6b',
                'secondary': '#ff8e72',
```

```
    },
    'ocean': {
        'primary': '#0099cc',
        'secondary': '#006699',
        'accent': '#00ccff',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'forest': {
        'primary': '#27ae60',
        'secondary': '#1e8449',
        'accent': '#52be80',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'sunset': {
        'primary': '#ff8c42',
        'secondary': '#ff6b35',
        'accent': '#ffa751',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'luxury': {
        'primary': '#1a1a1a',
        'secondary': '#333333',
        'accent': '#FFD700',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
```

```
        'creative': {
            'primary': '#9b59b6',
            'secondary': '#8e44ad',
            'accent': '#e74c3c',
            'background': '#0f172a',
            'surface': '#1e293b',
            'text_primary': '#f1f5f9',
            'text_secondary': '#cbd5e1',
            'border': '#334155'
        },
        'minimal': {
            'primary': '#2c3e50',
            'secondary': '#34495e',
            'accent': '#3498db',
            'background': '#0f172a',
            'surface': '#1e293b',
            'text_primary': '#f1f5f9',
            'text_secondary': '#cbd5e1',
            'border': '#334155'
        }
    }

    return palettes.get(style.lower(), palettes['modern'])
```

Activity 2.2: HTML Page Builder Implementation

The HTMLPageBuilder class includes:

- Section extraction from user descriptions
- Keyword detection for hero, features, about, portfolio, testimonials, pricing, contact, and footer sections
- SEO meta tags generation with Open Graph and Twitter Card support
- Individual section HTML generation with professional templates

```python
    @staticmethod
    def generate_meta_tags(title: str, description: str) -> str:
        """Generate SEO meta tags"""
        meta_tags = f'''    <meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{title}</title>
<meta name="description" content="{description[:160]}">
<meta name="theme-color" content="#0f172a">
<meta property="og:title" content="{title}">
<meta property="og:description" content="{description[:160]}">
<meta property="og:type" content="website">
<meta name="twitter:card" content="summary_large_image">
<meta name="author" content="HTML Quick-Styler">'''
        return meta_tags

    @staticmethod
    def generate_section_html(section_type: str, palette: Dict) -> str:
        """Generate HTML for specific section"""
        sections = {
            'hero': f'''        <section class="hero">
            <div class="hero-content">
                <h1>Welcome to Your Site</h1>
                <p>Create amazing web experiences</p>
                <button class="btn btn-primary">Get Started</button>
            </div>
        </section>''',

            'features': f'''        <section class="features">
            <h2>Features</h2>
            <div class="features-grid">
                <div class="feature-card">
                    <h3>Fast</h3>
                    <p>Lightning-quick performance</p>
                </div>
                <div class="feature-card">
                    <h3>Secure</h3>
                    <p>Enterprise-grade security</p>
                </div>
                <div class="feature-card">
                    <h3>Scalable</h3>
                    <p>Grow without limits</p>
                </div>
            </div>
        </section>''',

            'about': f'''        <section class="about">
            <div class="about-content">
                <h2>About Us</h2>
                <p>We're passionate about creating beautiful, functional websites that help your business grow.</p>
            </div>
        </section>''',

            'portfolio': f'''        <section class="portfolio">
            <h2>Our Work</h2>
            <div class="portfolio-grid">
                <div class="portfolio-item"></div>
                <div class="portfolio-item"></div>
                <div class="portfolio-item"></div>
            </div>
        </section>''',
```

```python
[ ]
class HTMLPageBuilder:
    """Build styled HTML pages from descriptions"""

    @staticmethod
    def extract_sections(description: str) -> List[str]:
        """Extract sections from description"""
        sections = []

        section_keywords = {
            'hero': ['hero', 'banner', 'landing'],
            'features': ['features', 'services', 'benefits'],
            'about': ['about', 'team', 'company'],
            'portfolio': ['portfolio', 'gallery', 'work', 'projects'],
            'testimonials': ['testimonial', 'reviews', 'feedback'],
            'pricing': ['pricing', 'plans', 'cost'],
            'contact': ['contact', 'form', 'reach'],
            'footer': ['footer', 'bottom']
        }

        desc_lower = description.lower()
```

```
        </section>''',

            'testimonials': f'''        <section class="testimonials">
            <h2>What People Say</h2>
            <div class="testimonials-grid">
                <div class="testimonial">
                    <p>"Amazing service!"</p>
                    <span>- Client Name</span>
                </div>
                <div class="testimonial">
                    <p>"Highly recommended!"</p>
                    <span>- Happy Customer</span>
                </div>
            </div>
        </section>''',

            'pricing': f'''        <section class="pricing">
            <h2>Pricing Plans</h2>
            <div class="pricing-grid">
                <div class="pricing-card">
                    <h3>Starter</h3>
                    <p class="price">$29</p>
                    <ul><li>Feature 1</li><li>Feature 2</li></ul>
                </div>
                <div class="pricing-card featured">
                    <h3>Pro</h3>
                    <p class="price">$79</p>
                    <ul><li>All Starter</li><li>Feature 3</li></ul>
                </div>
            </div>
        </section>''',

            'contact': f'''        <section class="contact">
            <h2>Get in Touch</h2>
            <form class="contact-form">
                <input type="text" placeholder="Your Name" required>
                <input type="email" placeholder="Your Email" required>
                <textarea placeholder="Your Message" rows="5" required></textarea>
                <button type="submit" class="btn btn-primary">Send Message</button>
            </form>
        </section>''',

            'footer': f'''        <footer class="footer">
            <div class="footer-content">
                <p>&copy; 2024 Your Company. All rights reserved.</p>
                <div class="footer-links">
                    <a href="#privacy">Privacy</a>
                    <a href="#terms">Terms</a>
                    <a href="#contact">Contact</a>
                </div>
            </div>
        </footer>'''
        }

        return sections.get(section_type, '')
```

Activity 2.3: CSS Generation Engine

The CSS generator creates comprehensive styling including:

- CSS variables for dynamic color management
- Gradient backgrounds and modern animations
- Responsive grid layouts with Flexbox and CSS Grid

- Smooth transitions and hover effects
- Mobile-first media queries for tablets and phones
- Sticky header with backdrop blur effect
- Card-based components with depth and shadows

```
        return sections.get(section_type,    )

    @staticmethod
    def generate_css(palette: Dict) -> str:
        """Generate comprehensive CSS"""
        css = f'''    <style>
        * {{
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }}

        :root {{
            --primary: {palette['primary']};
            --secondary: {palette['secondary']};
            --accent: {palette['accent']};
            --background: {palette['background']};
            --surface: {palette['surface']};
            --text-primary: {palette['text_primary']};
            --text-secondary: {palette['text_secondary']};
            --border: {palette['border']};
        }}

        body {{
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

PHASE 3: Gradio Interface Development

Activity 3.1: Main Interface with Multiple Tabs

The Gradio interface includes 5 tabs:

```
body {{
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, var(--background) 0%, var(--surface) 100%);
    color: var(--text-primary);
    line-height: 1.6;
    min-height: 100vh;
}}

header {{
    background: rgba(15, 23, 42, 0.95);
    backdrop-filter: blur(10px);
    padding: 1.5rem 2rem;
    position: sticky;
    top: 0;
    z-index: 100;
    border-bottom: 1px solid var(--border);
}}

header nav {{
    display: flex;
    justify-content: space-between;
    align-items: center;
    max-width: 1200px;
    margin: 0 auto;
}}

header h1 {{
    font-size: 1.8rem;
    background: linear-gradient(135deg, var(--primary), var(--accent));
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
}}

header nav a {{
    color: var(--text-secondary);
    text-decoration: none;
    margin: 0 1.5rem;
    transition: color 0.3s ease;
}}

header nav a:hover {{
    color: var(--primary);
}}

main {{
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 2rem;
}}

section {{
    padding: 4rem 0;
    animation: fadeIn 0.6s ease-in;
}}

@keyframes fadeIn {{
    from {{
        opacity: 0;
        transform: translateY(20px);
    }}
    to {{
        opacity: 1;
        transform: translateY(0);
    }}
}}

h1, h2, h3 {{
    color: var(--text-primary);
    margin-bottom: 1rem;
}}
```

```
h1 {{
    font-size: 3rem;
    font-weight: 700;
}}

h2 {{
    font-size: 2rem;
    margin-bottom: 2rem;
    text-align: center;
}}

h3 {{
    font-size: 1.3rem;
}}

.hero {{
    background: linear-gradient(135deg, var(--primary) 0%, var(--secondary) 100%);
    color: white;
    padding: 6rem 0;
    text-align: center;
    min-height: 500px;
    display: flex;
    align-items: center;
    justify-content: center;
}}

.hero h1 {{
    color: white;
    font-size: 4rem;
    margin-bottom: 1rem;
    animation: slideDown 0.8s ease;
}}

.hero p {{
    font-size: 1.5rem;
    margin-bottom: 2rem;
    opacity: 0.95;
    animation: slideUp 0.8s ease;
}}

@keyframes slideDown {{
    from {{
        opacity: 0;
        transform: translateY(-30px);
    }}
    to {{
        opacity: 1;
        transform: translateY(0);
    }}
}}

@keyframes slideUp {{
    from {{
        opacity: 0;
        transform: translateY(30px);
    }}
    to {{
        opacity: 1;
        transform: translateY(0);
    }}
}}
```

```
.btn {{
    padding: 0.8rem 2rem;
    border: none;
    border-radius: 8px;
    font-size: 1rem;
    cursor: pointer;
    transition: all 0.3s ease;
    font-weight: 600;
}}

.btn-primary {{
    background: var(--primary);
    color: white;
    box-shadow: 0 4px 15px rgba(0,0,0,0.3);
}}

.btn-primary:hover {{
    transform: translateY(-2px);
    box-shadow: 0 6px 20px rgba(0,0,0,0.4);
    background: var(--accent);
}}

.btn-secondary {{
    background: var(--secondary);
    color: white;
}}

.btn-secondary:hover {{
    background: var(--primary);
}}

.features-grid {{
    display: grid;
    grid-templ[illegible]lumns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 2rem;
}}

.feature-card {{
    background: rgba(30, 41, 59, 0.8);
    padding: 2rem;
    border-radius: 12px;
    border: 1px solid var(--border);
    transition: all 0.3s ease;
    backdrop-filter: blur(10px);
}}

.feature-card:hover {{
    transform: translateY(-8px);
    border-color: var(--primary);
    box-shadow: 0 20px 40px rgba(0,0,0,0.3);
}}

.feature-card h3 {{
    color: var(--primary);
    margin-bottom: 0.5rem;
}}

.feature-card p {{
    color: var(--text-secondary);
}}
```

```
.portfolio-grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 2rem;
}}

.portfolio-item {{
    aspect-ratio: 1;
    background: linear-gradient(135deg, var(--primary), var(--secondary));
    border-radius: 12px;
    cursor: pointer;
    transition: all 0.3s ease;
    border: 1px solid var(--border);
}}

.portfolio-item:hover {{
    transform: scale(1.05);
    box-shadow: 0 20px 40px rgba(0,0,0,0.3);
}}

.about {{
    text-align: center;
}}

.about-content {{
    max-width: 800px;
    margin: 0 auto;
}}

.about p {{
    font-size: 1.1rem;
    color: var(--text-secondary);
}}

.testimonials-grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 2rem;
}}

.testimonial {{
    background: rgba(30, 41, 59, 0.8);
    padding: 2rem;
    border-left: 4px solid var(--primary);
    border-radius: 8px;
}}

.testimonial p {{
    font-style: italic;
    margin-bottom: 1rem;
    color: var(--text-secondary);
}}

.testimonial span {{
    font-weight: 600;
    color: var(--primary);
}}

.pricing-grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 2rem;
}}
```

```
.pricing-card {{
    background: rgba(30, 41, 59, 0.8);
    padding: 2rem;
    border-radius: 12px;
    border: 1px solid var(--border);
    text-align: center;
    transition: all 0.3s ease;
}}

.pricing-card.featured {{
    border-color: var(--primary);
    box-shadow: 0 20px 40px rgba(0,0,0,0.3);
    transform: scale(1.05);
}}

.pricing-card .price {{
    font-size: 2.5rem;
    color: var(--primary);
    margin: 1rem 0;
    font-weight: 700;
}}

.pricing-card ul {{
    list-style: none;
    margin: 1.5rem 0;
    text-align: left;
}}

.pricing-card li {{
    padding: 0.5rem 0;
    color: var(--text-secondary);
}}

.contact {{
    max-width: 600px;
    margin: 0 auto;
}}

.contact-form {{
    display: flex;
    flex-direction: column;
    gap: 1.5rem;
}}

.contact-form input,
.contact-form textarea {{
    padding: 1rem;
    border: 1px solid var(--border);
    border-radius: 8px;
    background: rgba(30, 41, 59, 0.8);
    color: var(--text-primary);
    font-family: inherit;
    font-size: 1rem;
}}

.contact-form input:focus,
.contact-form textarea:focus {{
    outline: none;
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(0,0,0,0.3);
}}
```

```
        .footer-content {{
            max-width: 1200px;
            margin: 0 auto;
            padding: 0 2rem;
            display: flex;
            justify-content: space-between;
            align-items: center;
            flex-wrap: wrap;
            gap: 2rem;
        }}

        .footer-links {{
            display: flex;
            gap: 2rem;
        }}

        .footer-links a {{
            color: var(--text-secondary);
            text-decoration: none;
            transition: color 0.3s ease;
        }}

        .footer-links a:hover {{
            color: var(--primary);
        }}

        @media (max-width: 768px) {{
            header nav {{
                flex-direction: column;
                gap: 1rem;
            }}

            header h1 {{
                font-size: 1.4rem;
            }}

            .hero h1 {{
                font-size: 2rem;
            }}

            .hero p {{
                font-size: 1.1rem;
            }}

            h2 {{
                font-size: 1.5rem;
            }}

            main {{
                padding: 0 1rem;
            }}

            section {{
                padding: 2rem 0;
            }}

            .footer-content {{
                flex-direction: column;
                text-align: center;
            }}
        }}
    </style>'''
        return css
```

```python
        # Generate CSS
        css = HTMLPageBuilder.generate_css(palette)

        # Build complete HTML
        html = f'''<!DOCTYPE html>
<html lang="en">
<head>
{meta_tags}
{css}
</head>
<body>
    <header>
        <nav>
            <h1>{title}</h1>
            <div>
                <a href="#features">Features</a>
                <a href="#about">About</a>
                <a href="#contact">Contact</a>
            </div>
        </nav>
    </header>

    <main>
{sections_html}
    </main>
</body>
</html>'''

        return html, palette

generation_history = []

def generate_html_page(title: str, description: str, style: str = "modern") -> Tuple[str, str, str]:
    """Generate HTML page with styling"""
    if not title.strip() or not description.strip():
        return "", "❌ Please provide both title and description!", ""

    try:
        html_output, palette = HTMLGenerationEngine.generate_page(title, description, style)

        color_info = f"""
**Color Palette:**
- Primary: {palette['primary']}
- Secondary: {palette['secondary']}
- Accent: {palette['accent']}
- Background: {palette['background']}
"""

        generation_history.append({
            'title': title,
            'description': description,
            'html': html_output,
            'palette': palette,
            'timestamp': datetime.now().isoformat()
        })
```

```python
class HTMLGenerationEngine:
    """Main engine for HTML page generation"""

    @staticmethod
    def generate_page(title: str, description: str, style: str = "modern") -> Tuple[str, str]:
        """Generate complete HTML page"""

        # Generate color palette
        palette = ColorPaletteGenerator.generate_palette(style)

        # Extract sections
        sections = HTMLPageBuilder.extract_sections(description)

        # Generate meta tags
        meta_tags = HTMLPageBuilder.generate_meta_tags(title, description)

        # Generate sections
        sections_html = '\n\n'.join([
            HTMLPageBuilder.generate_section_html(section, palette)
            for section in sections
            if HTMLPageBuilder.generate_section_html(section, palette)
        ])
```

```
        return html_output, "✅ HTML page generated successfully!", color_info
    except Exception as e:
        return "", f"❌ Error: {str(e)}", ""

def get_color_suggestions(style: str) -> str:
    """Get color suggestions based on style"""
    palette = ColorPaletteGenerator.generate_palette(style)

    palette_text = f"""
**Palette for "{style}" style:**
- Primary: {palette['primary']}
- Secondary: {palette['secondary']}
- Accent: {palette['accent']}
- Background: {palette['background']}
- Text: {palette['text_primary']}
"""
    return palette_text

print("🚀 Loading HTML Quick-Styler Advanced...")

with gr.Blocks(title="HTML Quick-Styler - Advanced Edition") as demo:

    gr.Markdown("""
    # 🎨 HTML Quick-Styler Advanced
    ## Professional HTML Page Generator with Auto-Styling

    **Generate production-ready HTML pages in seconds**
    """)

    with gr.Tabs():
```

Tab 1: Generate Page

```
            )

            color_style = gr.Dropdown(
                ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
                value="modern",
                label="🎨 Color Style"
            )

            generate_btn = gr.Button("✨ Generate HTML", size="lg", variant="primary")

        with gr.Column(scale=1):
            gr.Markdown("### 📊 Output")

            html_output = gr.Code(
                language="html",
                label="Generated HTML/CSS",
                lines=20
            )

            status_msg = gr.Markdown()
            color_info = gr.Markdown()

    generate_btn.click(
        generate_html_page,
        inputs=[page_title, page_description, color_style],
        outputs=[html_output, status_msg, color_info]
    )
```

Tab 2: Live Preview

```
            outputs=[html_output, status_msg, color_info]
        )

    # ============= TAB 2: Preview =============
    with gr.TabItem("👁️ Live Preview"):
        gr.Markdown("### 🔍 Preview Generated Page")

        preview_button = gr.Button("🔄 Load Preview", size="lg", variant="primary")
        preview_output = gr.HTML(value="<p style='text-align:center; color:#fff; padding:40px;'>No preview loaded. Generate a page first.</p>")

        def show_preview(html_text):
            if not html_text.strip():
                return "<p style='text-align:center; color:#fff; padding:40px;'>No HTML provided</p>"
            return html_text

        preview_button.click(
            show_preview,
            inputs=[html_output],
            outputs=[preview_output]
        )

    # ============= TAB 3: Color Palettes =============
    with gr.TabItem("🎨 Color Palettes"):
        gr.Markdown("### 🌈 Explore Color Schemes")

        with gr.Row()
```

Tab 3: Color Palettes

```
            outputs=[preview_output]
        )

    # ============= TAB 3: Color Palettes =============
    with gr.TabItem("🎨 Color Palettes"):
        gr.Markdown("### 🌈 Explore Color Schemes")

        with gr.Row():
            style_selector = gr.Radio(
                ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
                value="modern",
                label="Choose Style"
            )

            palette_btn = gr.Button("🎨 Get Palette", size="lg", variant="primary")

        palette_display = gr.Markdown()

        palette_btn.click(
            get_color_suggestions,
            inputs=[style_selector],
            outputs=[palette_display]
        )
```

Tab 4: Templates

```python
# ============= TAB 4: Templates =============
with gr.TabItem("📋 Templates"):
    gr.Markdown("### 📝 Quick Templates")

    with gr.Row():
        with gr.Column():
            gr.Markdown("""
            #### Portfolio Website
            Hero, Portfolio Gallery, About, Contact, Footer
            """)
            portfolio_btn = gr.Button("Use Template", size="sm")

            def load_portfolio():
                return ("Portfolio Website",
                        "Hero section with welcome banner, portfolio gallery with 6 items, about section, testimonials, contact form, foot
                        "modern")

            portfolio_btn.click(load_portfolio, outputs=[page_title, page_description, color_style])

        with gr.Column():
            gr.Markdown("""
            #### SaaS Landing Page
            Hero, Features, Pricing, Testimonials, CTA, Footer
            """)
            saas_btn = gr.Button("Use Template", size="sm")
```

```python
            saas_btn = gr.Button("Use Template", size="sm")

            def load_saas():
                return ("SaaS Product",
                        "Hero banner, features showcase with 3 items, pricing plans comparison, testimonials, call-to-action button, footer",
                        "vibrant")

            saas_btn.click(load_saas, outputs=[page_title, page_description, color_style])

        with gr.Column():
            gr.Markdown("""
            #### Company Website
            Header, About, Services, Team, Contact, Footer
            """)
            company_btn = gr.Button("Use Template", size="sm")

            def load_company():
                return ("Company",
                        "Header with navigation, hero section, about company, services list, team members, contact form, footer",
                        "ocean")

            company_btn.click(load_company, outputs=[page_title, page_description, color_style])
```

Tab 5: Guide

```python
# ============= TAB 5: Guide =============
with gr.TabItem("📚 Guide"):
    gr.Markdown("""
    # 📖 HTML Quick-Styler Guide

    ## How It Works

    ### 1. Enter Details
    - **Title:** Page name
    - **Description:** Sections you want
    - **Color Style:** Pick a theme

    ### 2. Automatic Detection
    The tool recognizes these keywords:
    - `hero` / `banner` → Hero Section
    - `features` / `services` → Features Grid
    - `about` / `team` → About Section
    - `portfolio` / `gallery` → Portfolio
    - `testimonial` / `reviews` → Testimonials
    - `pricing` / `plans` → Pricing Plans
    - `contact` / `form` → Contact Form
    - `footer` → Footer
```

PHASE 4: Deployment

Activity 4.1: Google Colab Deployment

Launch the application in Google Colab:

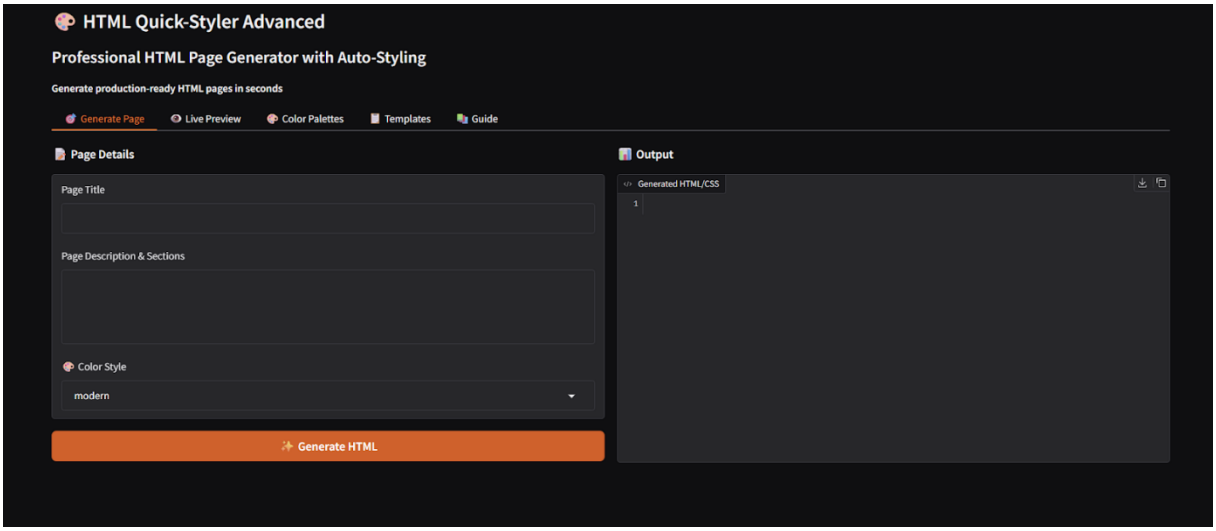
```
print("\n✅ HTML Quick-Styler Advanced Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

```
🚀 Loading HTML Quick-Styler Advanced...
ut actions
L Quick-Styler Advanced Ready!

/tmp/ipython-input-1159347522.py:956: DeprecationWarning: The 'show_api' parameter in launch() will be removed
  demo.launch(share=True, show_error=True, show_api=False)
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://87205c7f5a55c5adc8.gradio.live
```

The application will be accessible via:

- Public URL: https://a58b233b868172c6e4.gradio.live



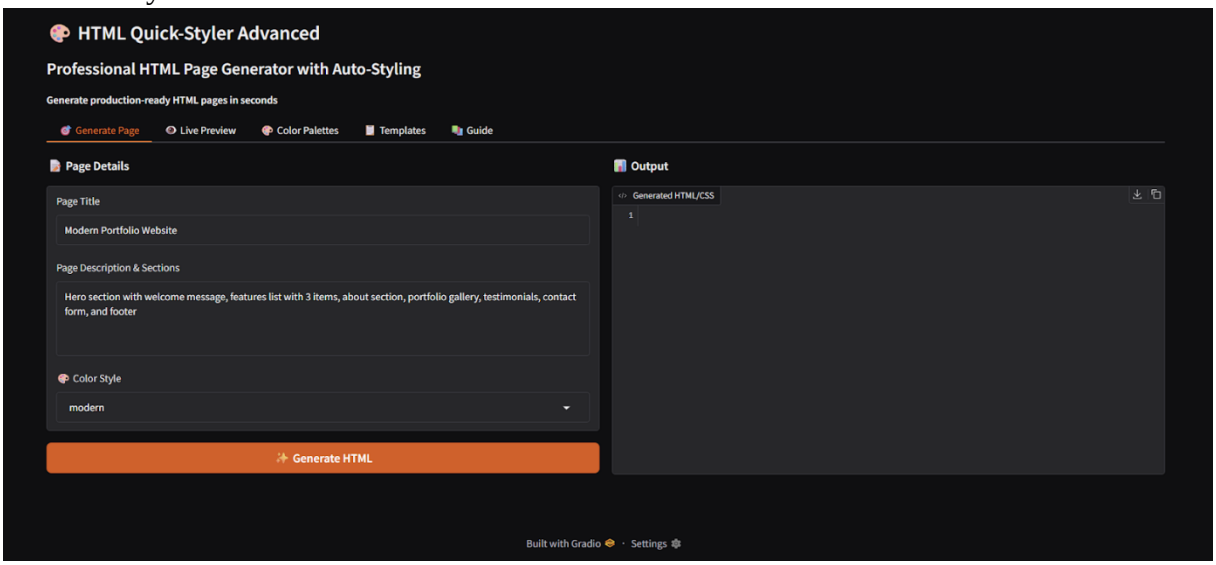
PHASE 5: Testing & Validation

Activity 5.1: Functional Testing

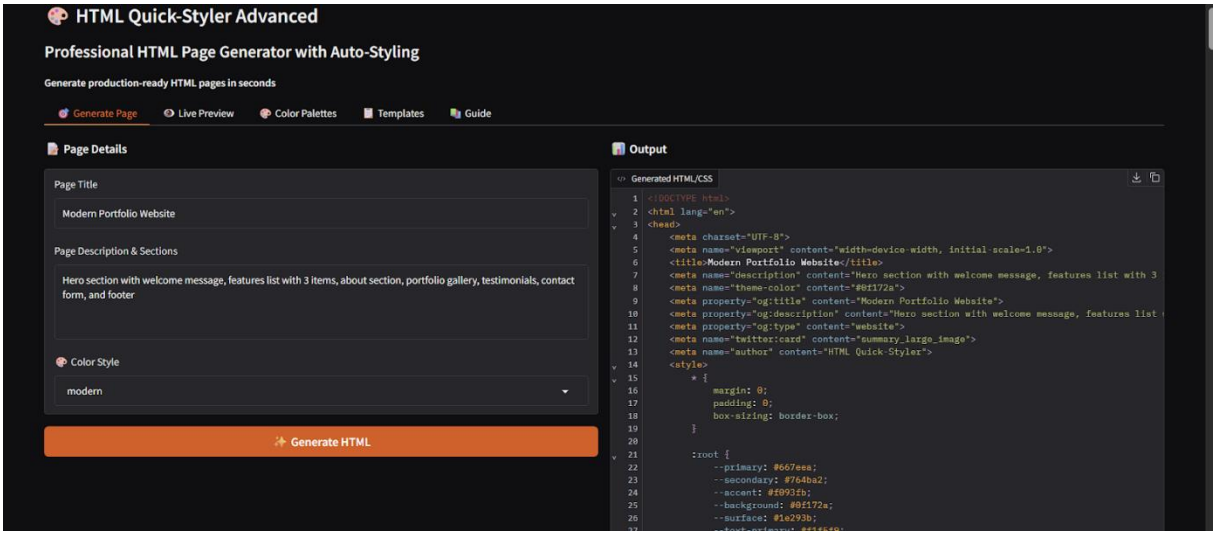
Test Case 1: Portfolio Website Generation

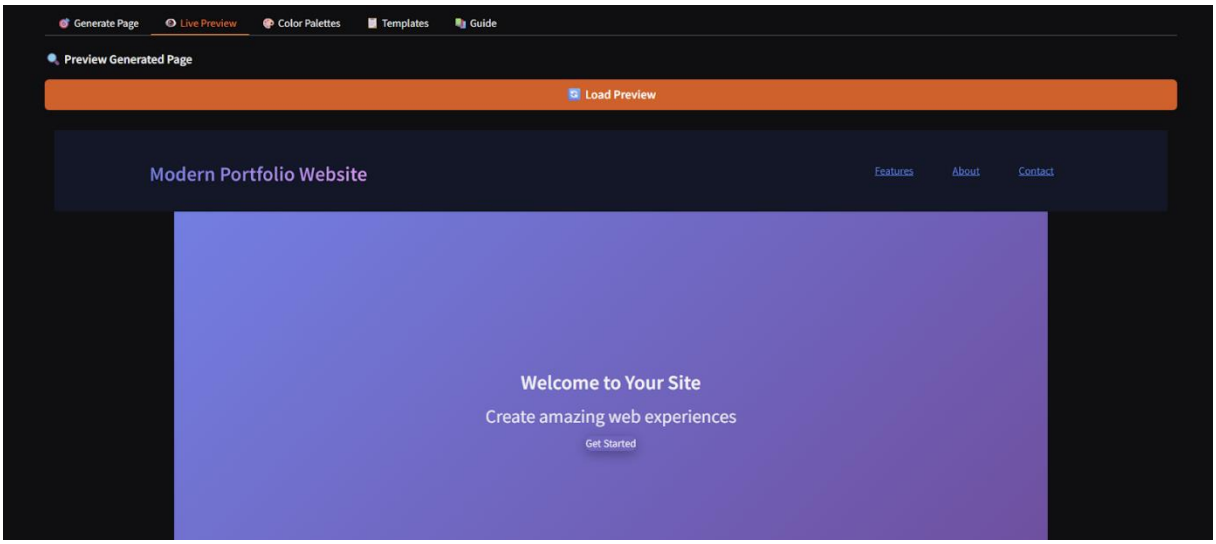
Input:

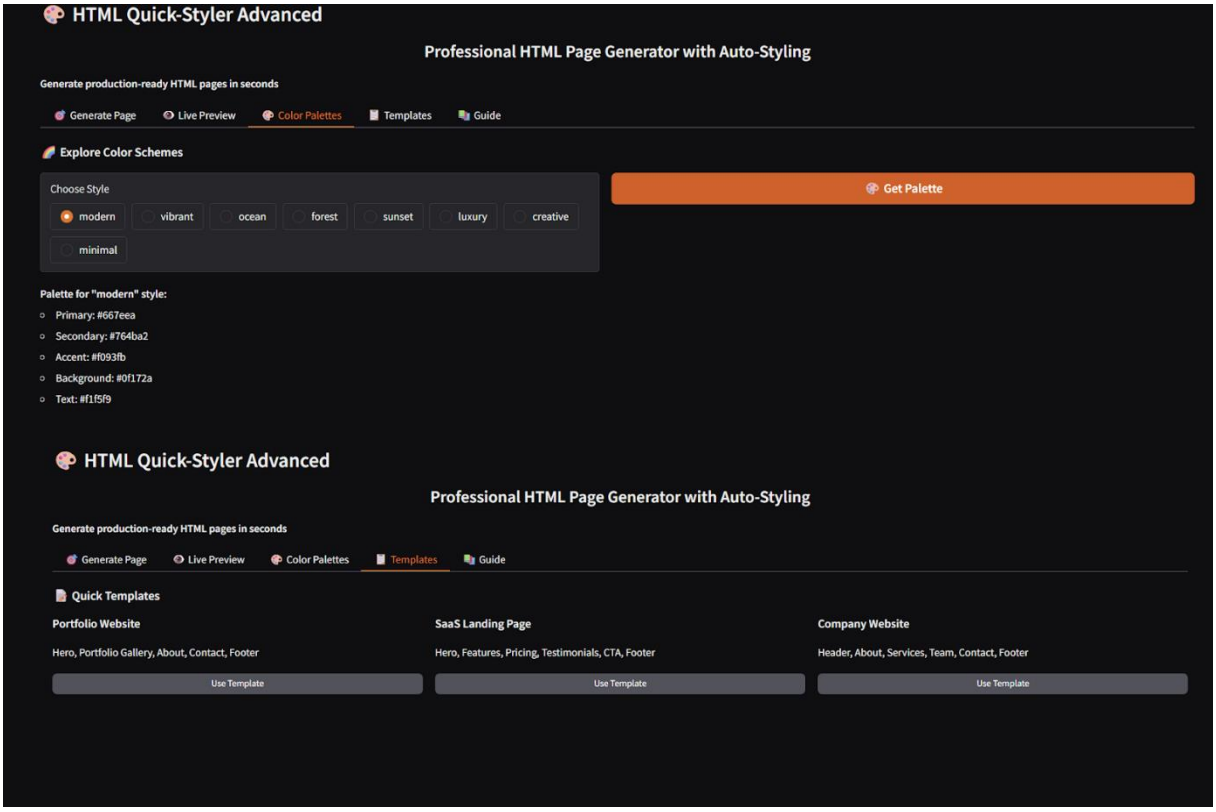
- Title: Modern Portfolio Website
- Description: Hero section with welcome message, features list with 3 items, about section, portfolio gallery, testimonials, contact form, and footer
- Color Style: modern

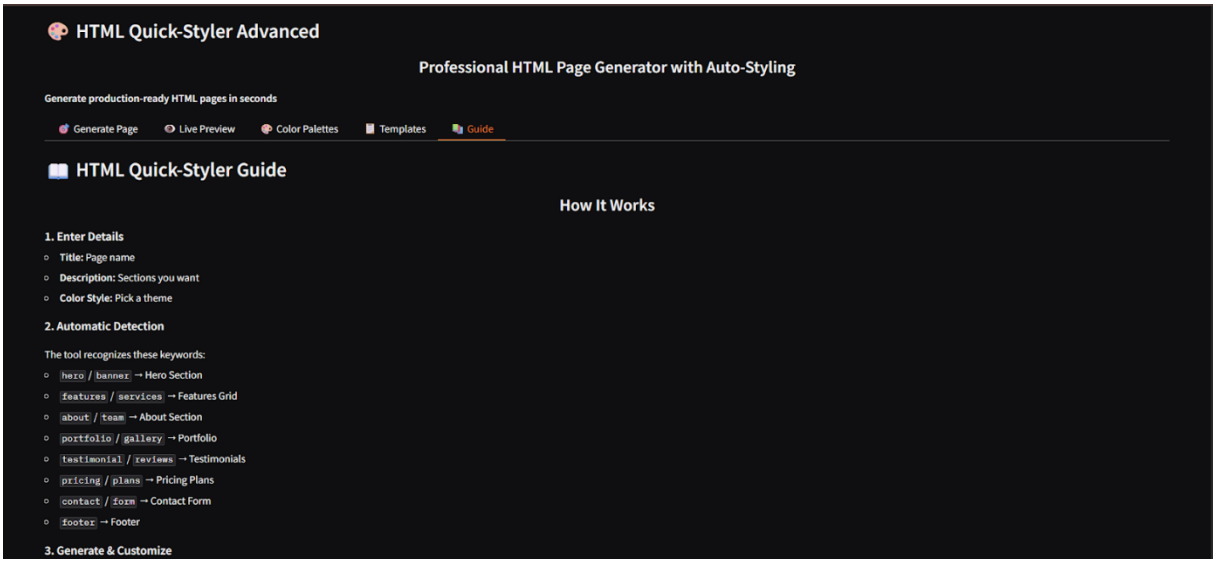


Expected Output:




Generate Page
Live Preview
Color Palettes
Templates
Guide
Preview Generated Page
Load Preview
Modern Portfolio Website
Features
About
Contact
Welcome to Your Site
Create amazing web experiences
Get Started


HTML Quick-Styler Advanced
Professional HTML Page Generator with Auto-Styling
Generate production-ready HTML pages in seconds
Generate Page
Live Preview
Color Palettes
Templates
Guide
Explore Color Schemes
Choose Style
modern
vibrant
ocean
forest
sunset
luxury
creative
minimal
Get Palette
Palette for "modern" style:
Primary: #667eea
Secondary: #764ba2
Accent: #f093fb
Background: #0f172a
Text: #f1f5f9
HTML Quick-Styler Advanced
Professional HTML Page Generator with Auto-Styling
Generate production-ready HTML pages in seconds
Generate Page
Live Preview
Color Palettes
Templates
Guide
Quick Templates
Portfolio Website
Hero, Portfolio Gallery, About, Contact, Footer
Use Template
SaaS Landing Page
Hero, Features, Pricing, Testimonials, CTA, Footer
Use Template
Company Website
Header, About, Services, Team, Contact, Footer
Use Template


HTML Quick-Styler Advanced
Professional HTML Page Generator with Auto-Styling
Generate production-ready HTML pages in seconds
Generate Page
Live Preview
Color Palettes
Templates
Guide
HTML Quick-Styler Guide
How It Works
1. Enter Details
Title: Page name
Description: Sections you want
Color Style: Pick a theme
2. Automatic Detection
The tool recognizes these keywords:
hero / banner → Hero Section
features / services → Features Grid
about / team → About Section
portfolio / gallery → Portfolio
testimonial / reviews → Testimonials
pricing / plans → Pricing Plans
contact / form → Contact Form
footer → Footer
3. Generate & Customize

CONCLUSION

Conclusion

HTML Quick-Styler is developed as an intelligent web page generation assistant that simplifies the process of creating modern and professional websites. By integrating the capabilities of the IBM Granite 3.3 2B Instruct model with the interactive features of the Gradio interface and the cloud-based environment of Google Colab, the system enables fast and efficient generation of HTML and CSS code. The platform allows designers, developers, and even non-technical users to generate well-structured web pages without requiring advanced programming knowledge.

The system demonstrates how artificial intelligence can support and enhance the web development process. By providing ready-to-use design templates, responsive layouts, and production-ready code, HTML Quick-Styler helps users create visually appealing websites with minimal effort. The inclusion of multiple color palette options, automatic section generation, and customizable templates makes the platform flexible and suitable for different types of web projects.

Overall, HTML Quick-Styler proves that AI-assisted tools can significantly reduce the time required for website development while maintaining professional design quality and usability standards.

In the future, the system can be further improved by introducing advanced features such as custom color palette selection, automatic JavaScript generation for interactive components, an expanded library of design templates, and integration with modern web development frameworks. The modular structure of the system makes it easier to extend and upgrade, allowing the platform to adapt to new technologies and evolving design trends.